

1. Core State Integrity (This is the root problem)

Problem

- Users are not consistently in the same system state
- Example: “one user is not in the main group state”
- You had to leave and rejoin to fix it

What this means

Your state machine is broken. The system allows invalid states.

Fix

- Enforce strict states:
 - registered
 - lobby
 - main room
 - breakout room
 - completed
- One user = one state at all times
- No exceptions

Also

- On join → force user into correct state
 - On reconnect → restore correct state from backend
 - No manual reset via “leave and rejoin”
-

2. Multi-Login / Duplicate Identity Issue

Problem

- You joined as:
 - Stefan Avivson
 - Mister Raw
 - Vox
 - Private
- System treats them as separate users

Impact

- Breaks matching
- Breaks room allocation
- Breaks counts

Fix

- One account = one event presence
 - Detect duplicates by:
 - user_id (not name)
 - Block multiple joins from same account
 - Or clearly separate test mode vs real users
-

3. Rematch Button Is Fake (Functionally Broken)

Problem

- Rematch runs but returns identical matches
- No visible change
- From your perspective: it does nothing

Root cause

- Same input → same deterministic output

Fix options (choose one explicitly)

1. Add randomness seed on rematch
2. Force “exclude previous matches” on rematch
3. Allow host to trigger:
 - full reset of matching history for that round

Expected behavior

Pressing “rematch” must produce different pairings. Always.

4. Matching Engine Not Completing Assignments

Problem

- “Two people are unmatched”
- With 6 people → should always fully match

Impact

- Dead users
- Broken experience

Fix

- Matching must guarantee:
 - all users assigned OR
 - explicit fallback (3-person room)

Rule

No user should ever be left unmatched unless explicitly shown why.

5. Matching Logic Not Optimized for Multi-Round Events

Problem

- You said it yourself:
 - everyone should meet new people across rounds
- System does not enforce this

Fix

- Add constraint:
 - minimize repeat matches within event
- Track:
 - who met who within event (only per event)

Also missing

- No fairness logic (new vs returning users)
-

6. Manual Room Creation Is Broken / Inconsistent

Problems

- Button sometimes disabled incorrectly
- Clicking didn't work
- Then suddenly worked

Fix

- Remove hidden conditions
- Button state must reflect:
 - always available if host
 - independent of previous rounds

Also

- System must confirm action:

- “Rooms created”
 - “Users moved”
-

7. Users Not Moving to Rooms

Problem

- Rooms created → users not moved

Critical failure

This kills the product.

Fix

- Room creation must:
 1. Create rooms
 2. Assign users
 3. Move users
- All three must be atomic (one operation)

If any fail → rollback and show error

8. Chat System Broken (Backend → Frontend mismatch)

Problem

- Message received notification appears
- Message itself not shown

Meaning

- Backend works
- Frontend render fails

Fix

- Validate message payload format
 - Ensure frontend listens to correct event
 - Add fallback:
 - if message count > 0 → force fetch messages
-

9. UI State Not Syncing with Backend

Examples

- Registered but UI doesn't show it
- Matches exist but not visible
- Rematch happens but UI unchanged

Fix

- After every action:
 - force UI refresh from backend state
 - Stop relying on optimistic UI
-

10. Event Flow Logic Is Broken

Problems

- "4 rounds out of 3 total"
- Cannot create new round properly
- Round count inconsistent

Fix

- Define clear event structure:
 - total rounds
 - current round index
 - Prevent impossible states
-

11. Metrics / Stats Are Wrong

Problems

- "You met 0 people" (incorrect)
- "12 mutual matches" (nonsense)
- Host stats incorrect

Fix

- Rebuild metrics from actual event logs
- Define clearly:
 - what is a "match"
 - what is a "mutual match"

Right now it's just printing garbage numbers

12. Host Control Is Unclear and Incomplete

Problems

- No clarity on:
 - who is matched
 - who is not
- No control over:
 - forcing match
 - excluding users

Fix

Host dashboard must show:

- all users
- their state
- their match status

And allow:

- manual override
-

13. System Requires Manual Reset to Work

Problem

- “leave and rejoin” fixes things

This is unacceptable

Fix

- System must self-heal:
 - on state mismatch → auto-correct
 - Never rely on user workaround
-

14. Logging Is There but Not Used for UX

You have:

- Sentry
- logs

But user sees nothing

Fix

- Surface errors to host:
 - “2 users failed to match”
 - “chat failed to load”
 - Silent failure = broken trust
-

What You’re Actually Missing (Strategically)

Right now you built:

A system that *tries* to run

What you need:

A system that *guarantees outcomes*

That means:

- deterministic state control
 - guaranteed matching completion
 - visible system truth to host
 - no silent failures
-

If I reduce this to 3 priorities

If Ali fixes only three things, fix these first:

1. **State machine (single source of truth for every user)**
2. **Matching completion (no unmatched users, no duplicates)**
3. **UI sync with backend (what happens is visible immediately)**

Everything else is secondary.